

1. Squares of a Sorted Array

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of the squares of each number sorted in non-decreasing order*.

Example 1:

Input: `nums = [-4,-1,0,3,10]`

Output: `[0,1,9,16,100]`

Explanation: After squaring, the array becomes `[16,1,0,9,100]`.

After sorting, it becomes `[0,1,9,16,100]`.

Example 2:

Input: `nums = [-7,-3,2,3,11]`

Output: `[4,9,9,49,121]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` is sorted in **non-decreasing** order.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an $O(n)$ solution using a different approach?

PROGRAM

```
class Solution {
```

```
    public int[] sortedSquares(int[] nums) {
```

```
        int n = nums.length;
```

```
        int[] result = new int[n];
```

```
        int left = 0;
```

```
        int right = n - 1;
```

```
        // Fill result from the largest square to smallest
```

```
        for (int i = n - 1; i >= 0; i--) {
```

```
            int leftSquare = nums[left] * nums[left];
```

```
            int rightSquare = nums[right] * nums[right];
```

```
if (leftSquare > rightSquare) {
```

```
    result[i] = leftSquare;
```

```
    left++;
```

```
} else {
```

```
    result[i] = rightSquare;
```

```
    right--;
```

```
}
```

```
}
```

```
return result;
```

```
}
```

```
}
```

OUTPUT

The screenshot displays a LeetCode submission for the problem "Squares of a Sorted Array". The submission is marked as "Accepted" with 137/137 test cases passed. The user's performance is shown as 2 ms (Beats 42.45%) and 46.96 MB (Beats 93.86%). The code is written in Java and implements a two-pointer approach to find the squares of a sorted array. The test result shows the input [-4, -1, 0, 3, 10] and the output [0, 1, 9, 16, 100].

```
1 class Solution {
2     public int[] sortedSquares(int[] nums) {
3
4         int n = nums.length;
5         int[] result = new int[n];
6
7         int left = 0;
8         int right = n - 1;
9
10        // Fill result from the largest square to smallest
11        for (int i = n - 1; i >= 0; i--) {
```

Runtime: 2 ms | Beats 42.45%
Memory: 46.96 MB | Beats 93.86%

Testcase: Case 1, Case 2
Input: nums = [-4, -1, 0, 3, 10]
Output: [0, 1, 9, 16, 100]
Expected: [0, 1, 9, 16, 100]